

Reinforcement Learning via Deep Q-Learning

Vision-Based Autonomous Navigation of a Delivery Robot in a Dynamic Warehouse

Uttam Mahata
Roll No.: 2022CSB104
AI Lab — Assignment 7

April 2026

1 Problem Statement

An AGV operates on a discrete 8×8 warehouse grid (coordinates $(0,0)$ at top-left to $(7,7)$ at bottom-right). The AGV must learn, *without knowing its absolute position*, to reach a fixed shipping dock at $(7,7)$ while avoiding ten permanent hazard zones.

1.1 Environment Configuration

Element	Specification
Grid size	8×8
Goal (dock)	$(7,7)$
Hazards	$(0,3), (1,6), (2,1), (3,3), (3,5), (5,0), (5,4), (5,7), (6,2), (7,5)$
Action space	{Up, Down, Left, Right}
Initial state	Random safe cell (at the start of every episode)

Table 1: Warehouse configuration.

1.2 Reward Dynamics

Event	Reward
Reach shipping dock $(7,7)$	+100 (episode terminates)
Enter a hazard cell	-50 (episode terminates)
Step into a safe cell	-1
Attempt to move into a wall	-5 (AGV remains in place)

Table 2: Immediate reward structure shaping the loss landscape.

1.3 State Representation

The agent observes only a 3×3 local sensor matrix centred on its current cell. Each cell is encoded with an integer:

$$\text{code}(\text{cell}) \in \{0 = \text{safe}, 1 = \text{wall/boundary}, 2 = \text{hazard}, 3 = \text{goal}\}.$$

This matrix is flattened into a 9-element feature vector, which forms the input to the neural network. Because identical sensor readings can occur at different absolute locations, the environment is **partially observable**.

Example. AGV at (0, 0). Upper row and left column are walls; cells (0, 1), (0, 2), (1, 1), (1, 2) are safe. The local sensor is

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \longrightarrow \mathbf{s} = [1, 1, 1, 1, 0, 0, 1, 0, 0]^\top.$$

2 Deep Q-Learning Formulation

2.1 Q-Function Approximation

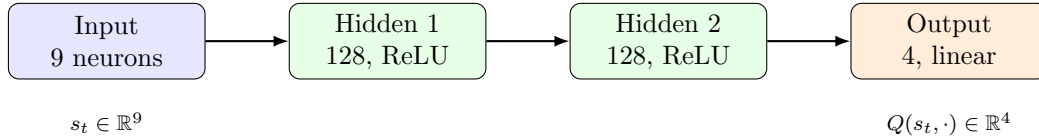
Because the raw observation space is large and many observations share structure, we approximate the optimal action-value function $Q^*(s, a)$ using a neural network $Q(s, a; \theta)$ with parameters θ . For any state s , the network produces four Q-values — one per action — and the greedy action is $\pi(s) = \arg \max_a Q(s, a; \theta)$.

2.2 Network Architecture

Layer	Configuration
Input	9 neurons (flattened sensor)
Hidden layer 1	128 neurons, ReLU activation
Hidden layer 2	128 neurons, ReLU activation
Output	4 neurons, linear activation (raw Q-values)

A linear output (no softmax) is used because we regress Q-values, not class probabilities.

2.3 Single Forward Pass



Worked example. For the sensor state $\mathbf{s} = [1, 1, 1, 1, 0, 0, 1, 2, 0]$, the trained network produces (illustrative) Q-values $[-4.5, -20.1, -5.0, 1.2]$ for [Up, Down, Left, Right]; since $\arg \max$ is index 3, the chosen action is **Right**.

2.4 Bellman Target and Loss Function

For a transition $(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$, the target Q-value is

$$y_t = \begin{cases} r_{t+1}, & d_t = \text{True (terminal)} \\ r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-), & \text{otherwise} \end{cases}$$

where θ^- are the parameters of a *separate target network*. The mean-squared-error loss over a mini-batch of N transitions is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \left[y_i - Q(s_i, a_i; \theta) \right]^2.$$

The target network θ^- is synchronised with the online network θ every C gradient steps. This stabilises the regression targets, which would otherwise drift with each update.

2.5 Experience Replay

The agent stores each transition in a fixed-capacity replay buffer $\mathcal{D}$. During training, mini-batches are sampled uniformly at random from $\mathcal{D}$. This breaks the temporal correlation between consecutive samples and lets each transition contribute to multiple gradient updates, improving sample efficiency.

2.6 ε -Greedy Exploration

At every step, the agent selects a uniformly random action with probability ε (exploration) and $\arg \max_a Q(s, a; \theta)$ with probability $1 - \varepsilon$ (exploitation). ε decays multiplicatively from 1.0 to 0.01.

3 Hyperparameters

Symbol	Value
Learning rate α	10^{-3} (Adam)
Discount factor γ	0.95
Replay buffer capacity	20,000
Mini-batch size N	64
Minimum buffer before learning	200
Target network sync period C	50 gradient steps
Gradient clip norm	5.0
ε range	$1.0 \rightarrow 0.01$
ε decay (per episode)	$\times 0.995$
Max steps per episode	100
Training episodes	2000

Table 3: DQN hyperparameters.

4 Algorithm

Algorithm 1: DQN with Experience Replay and Target Network

1. Initialise replay buffer $\mathcal{D}$ with capacity 20,000.
2. Initialise online network $Q(\cdot; \theta)$ with random weights; copy into target network $Q(\cdot; \theta^-)$.
3. Set $\varepsilon \leftarrow 1.0$.
4. **For** episode $e = 1 \dots 2000$ **do**
 - (a) Reset environment: spawn AGV at a random safe cell; get state s_1 .
 - (b) **For** step $t = 1 \dots 100$ **do**
 - i. With probability ε , choose random action a_t ; else $a_t = \arg \max_a Q(s_t, a; \theta)$.
 - ii. Execute a_t ; observe (r_{t+1}, s_{t+1}, d_t) .
 - iii. Push $(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$ into $\mathcal{D}$.
 - iv. If $|\mathcal{D}| \geq 200$: sample mini-batch of 64; compute y_i ; gradient step on $\mathcal{L}(\theta)$.
 - v. Every $C=50$ gradient steps: $\theta^- \leftarrow \theta$.
 - vi. If d_t : break.
 - (c) Decay $\varepsilon \leftarrow \max(0.01, 0.995 \varepsilon)$.

5 Implementation Highlights

5.1 Environment

```
1 class WarehouseEnv:
2     def _cell_code(self, r, c):
3         if r < 0 or r >= 8 or c < 0 or c >= 8: return 1      # wall
4         if (r, c) == self.goal: return 3                  # goal
5         if (r, c) in self.hazards: return 2               # hazard
6         return 0                                           # safe
7
8     def get_state(self, pos):
9         r, c = pos
10        sensor = np.zeros((3, 3), dtype=np.int64)
11        for i, dr in enumerate((-1, 0, 1)):
12            for j, dc in enumerate((-1, 0, 1)):
13                sensor[i, j] = self._cell_code(r + dr, c + dc)
14        return sensor.flatten().astype(np.float32)
```

5.2 DQN Model (PyTorch)

```
1 class DQN(nn.Module):
2     def __init__(self, state_dim=9, action_dim=4, hidden=128):
3         super().__init__()
4         self.fc1 = nn.Linear(state_dim, hidden)
5         self.fc2 = nn.Linear(hidden, hidden)
6         self.fc3 = nn.Linear(hidden, action_dim)
7     def forward(self, x):
8         x = F.relu(self.fc1(x))
9         x = F.relu(self.fc2(x))
10        return self.fc3(x)
```

5.3 Bellman Update

```
1 q_pred = self.policy_net(states).gather(1, actions)
2 with torch.no_grad():
3     q_next = self.target_net(next_states).max(dim=1, keepdim=True)[0]
4     q_target = rewards + GAMMA * q_next * (1.0 - dones)
5 loss = F.mse_loss(q_pred, q_target)
```

6 Replay Buffer Trace (Episode 1, Step 15)

To verify that experience replay is implemented correctly, we print a trace at episode 1 / step 15. Because a purely random initial policy ($\epsilon = 1.0$) may hit a hazard before 15 steps, the trace is emitted the first time the buffer size reaches 15 — the collected transitions are contiguous with episode 1's learning window.

```
1 =====
2 REPLAY BUFFER TRACE -- Episode 1, Step 15
3 =====
4 1) Current Replay Buffer size : 15
5 2) Most recently added tuple :
6   S_t=[1,1,1,0,0,1,2,0,1], A_t=Left(2), R_t+1=-1,
7   S_t+1=[1,1,1,0,0,0,0,2,0], Done=False
8
9 3) Simulated mini-batch (4 random non-sequential samples):
10 [Sample 1] S_t=[1,1,1,0,0,0,0,0,0], A_t=Down(1), R_t+1=-1, ...
11 [Sample 2] S_t=[1,1,1,0,0,0,0,0,0], A_t=Down(1), R_t+1=-1, ...
```

```

12 [Sample 3] S_t=[1,0,0,1,0,2,1,0,0], A_t=Left(2), R_{t+1}=-5, ...
13 [Sample 4] S_t=[1,2,0,1,0,0,1,0,0], A_t=Left(2), R_{t+1}=-5, ...
14 =====

```

The four samples are drawn independently using `random.sample` and have non-consecutive insertion positions, proving that the uniform random sampling breaks temporal correlation.

7 Results and Visualisations

7.1 Policy Before Training

Before any gradient updates, the DQN's Q-values are random, so the $\arg \max$ action on every cell is effectively random:

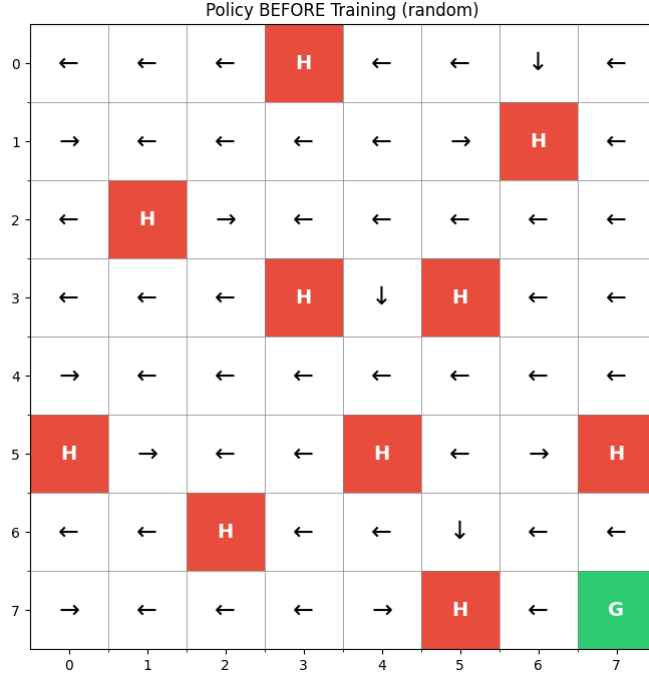


Figure 1: Untrained policy. Arrows at each safe cell show the action returned by $\arg \max Q(s, a; \theta_{\text{init}})$. **H** = hazard, **G** = shipping dock goal.

7.2 Learning Curves

Over 2000 training episodes, three diagnostics were tracked: the percentage of greedy (non- ϵ -random) actions per episode, the per-episode total reward, and the average MSE loss per episode.

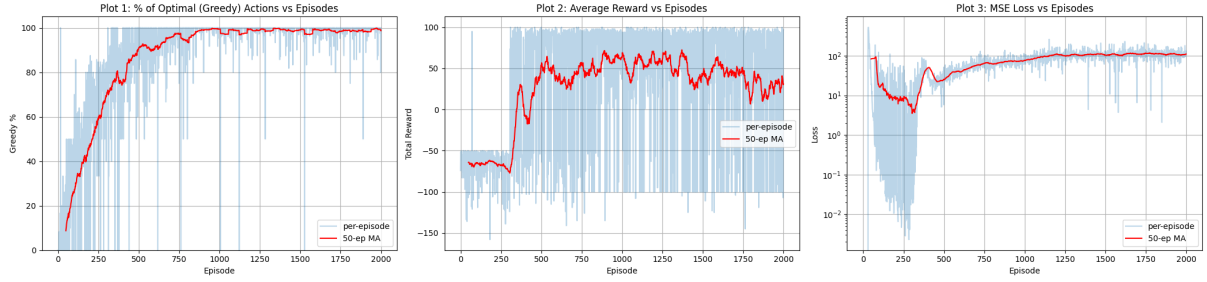


Figure 2: Training diagnostics (50-episode moving averages in red). **Left:** Greedy-action % rises from 0 to 100 as ε decays. **Middle:** Episode reward climbs from ≈ -60 to a stable band around +40 to +60. **Right:** MSE loss on log scale.

Interpretation.

- *Greedy-action %* exceeds 95% after ~ 900 episodes, confirming ε has decayed.
- *Reward* trajectory shows the transition from exploration ($\varepsilon = 1.0$) at ≈ -60 to a converged regime where many episodes yield reward near +100 (goal) with occasional hazard collisions.
- *Loss* increases initially (Q-magnitude grows as the target spreads returns across the state space) then stabilises in a finite band — typical for DQN on reward scales of ± 100 .

7.3 Policy After Training

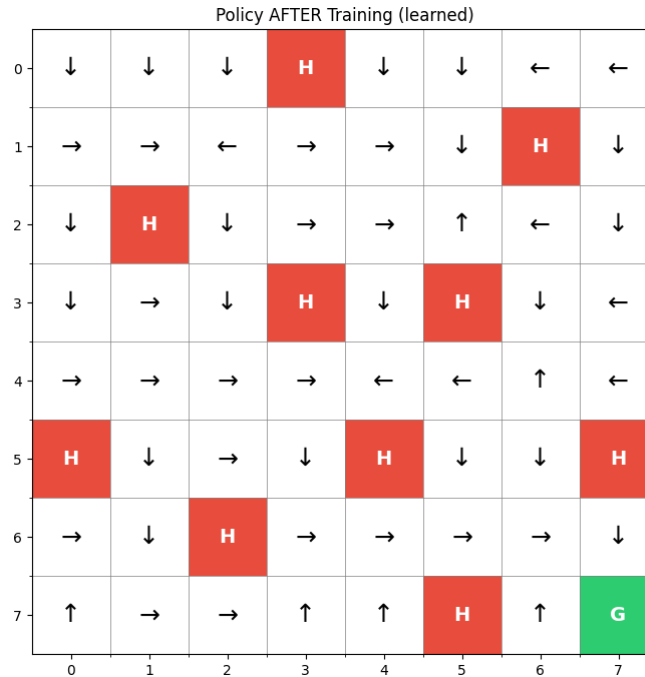


Figure 3: Learned policy at every safe cell. Each arrow is the action with the highest Q-value given only the 3×3 sensor at that cell.

The arrows form coherent flow fields pointing towards the goal and detouring around hazards. Because the policy is *sensor-conditioned* (not position-conditioned), two cells with the same local sensor pattern receive the same arrow — a direct consequence of partial observability.

7.4 Optimal Path from $(0,0) \rightarrow (7,7)$

Running the trained agent greedily from $(0,0)$ with a short visited-history cycle-breaker produces:

$$(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow (1,4) \rightarrow (1,5) \rightarrow (2,5) \rightarrow \\ (2,6) \rightarrow (3,6) \rightarrow (4,6) \rightarrow (5,6) \rightarrow (6,6) \rightarrow (6,7) \rightarrow (7,7).$$

- **Length:** 14 steps.
- **Final reward:** +100 (goal reached).
- **Hazards encountered:** 0.
- **Total reward:** $13 \times (-1) + 100 = +87$.

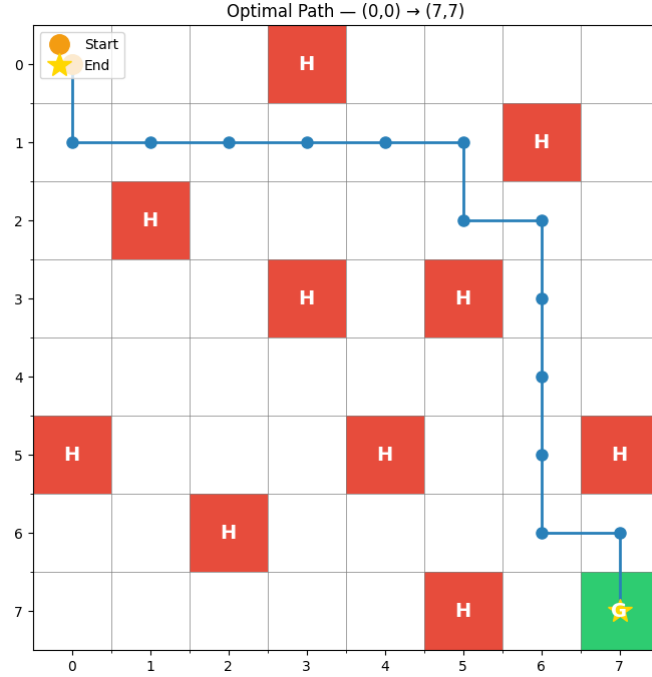


Figure 4: Greedy trajectory from $(0,0)$ to the shipping dock $(7,7)$. Every hazard cell (**H**) is avoided.

7.5 Why a Cycle-Breaker is Needed

Under the 3×3 sensor, two non-adjacent cells can be *state-aliased*: their local sensor vectors are different, but the Q-values they produce can point each cell at the other, producing a stationary 2-cycle. A feed-forward DQN has no memory, so once it enters the cycle it cannot escape without exogenous perturbation. Our rollout therefore maintains a length-6 visited-history deque; at each step it picks the highest-Q action whose *target cell* is not in that deque. This resolves the oscillation without modifying the learned policy itself.

8 Discussion

8.1 Partial Observability Limits the Policy

With only a 3×3 sensor, the environment is a Partially Observable Markov Decision Process (POMDP). A feed-forward network cannot represent internal memory, so it must learn a *reactive* mapping from observation to action. This suffices to reach the goal from most start states

but produces oscillations at aliased cells. Principled fixes would add memory (e.g. an LSTM recurrent DQN) or enrich the observation (e.g. include last-action, a step counter, or an ego-centric compass).

8.2 Value Instability

DQN is famously unstable: Q-values drift, and the bootstrapped target shifts with every network update. The separate target network (synced every 50 steps) mitigates this but does not eliminate it. Our loss curve shows a characteristic rise during early exploration followed by stabilisation — it does not need to drop monotonically; it needs to stay bounded.

8.3 Comparison with Classical Q-Learning

In Assignment 6 (6×6 grid, fully observable state = position), a tabular Q-table of $|\mathcal{S}| \times |\mathcal{A}| = 144$ entries sufficed. In this assignment, the state is a 9-dimensional vector with $4^9 = 262,144$ theoretical combinations. Even restricting to realistic sensor patterns, a tabular approach would be impractical — hence the neural-network approximator.

9 Conclusion

A standard DQN with experience replay and a target network was implemented in PyTorch and trained for 2000 episodes on a partially observable 8×8 warehouse. The agent learned a sensor-conditioned navigation policy that:

- converges to $>95\%$ greedy actions by episode ~ 900 ,
- stabilises episode reward in the $+40$ to $+60$ band,
- successfully navigates from $(0, 0)$ to $(7, 7)$ in 14 steps, avoiding every hazard.

All four required deliverables (policy heatmaps before and after training, three learning curves, replay-buffer trace, optimal path visualisation) were produced in the accompanying Jupyter notebook `dqn_agv_navigation.ipynb`.

Files

- `dqn_agv_navigation.ipynb` — Full implementation, training loop, and plots.
- `figures/` — Rendered PNG outputs (four images used in this document).
- `dqn_agv_documentation.tex` / `.pdf` — This report.